V0 ONTOLOGICAL AND STRUCTURAL-ACTIVATION PROGRAM

v1.1

FORMAL CORE, CANONICAL SCHEMAS, AND PROOF-ASSISTANT MAPPING

A normative specification of the smallest invariant core, machine-readable registry and proof schemas, deterministic serialization, diagnostic precedence, and correspondence targets for Lean 4 and Coq

EVIDENCE STATUS Specification baseline and schema package. JSON schemas are locally validated. Lean 4 and Coq mappings are design-level correspondences and are not claimed compiler-passed in this release.

TYPE BEFORE RULE - STATE BEFORE PROOF - HASH IS TRACEABILITY, NOT TRUTH

Dmytro Panasenko

Independent Researcher | 10 July 2026

Lineage: v1.0 integrated baseline -> v1.1 formal-core and mechanization specification

0. Document control and normative status

0.1 Version identity

The artifact identifier is V0_OSAP_v1_1. It refines the v1.0 integrated baseline into a machine-oriented specification for the smallest invariant core. It does not alter AX0-AX7, add physical dynamics, or claim proof-assistant verification. Its purpose is to make the accepted fragment explicit enough for independent implementations, schema validation, fixture construction, and later theorem-prover replay.

0.2 Normative artifacts

Artifact	Role	Status
Main specification	Human-readable normative definition of the core and mappings.	Normative v1.1 baseline.
Canonical schema bundle	JSON Schema definitions, examples, mapping notes, and manifest.	Machine-readable; locally schema-validated.
Proof-assistant mapping	Lean 4 and Coq type/rule correspondence tables.	Design mapping; not compiler-passed.
Preflight and SHA-256	Layout, package, and byte-identity evidence.	Release support only; not semantic proof.

0.3 Normative keywords

MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, MAY, PASS, REJECT, DEFERRED, INDETERMINATE, OUT-OF-SCOPE, SPECIFIED, SCHEMA-VALID, MAPPED, and COMPILER-PASSED are status tokens. SPECIFIED, SCHEMA-VALID, MAPPED, and COMPILER-PASSED are deliberately distinct.

0.4 Freeze and amendment rule

AX0-AX7 and the v1.0 integrated semantic firewalls remain frozen. v1.1 may narrow the accepted fragment and add representation constraints, but it may not weaken domain guards, historical-token separation, residual obstruction, absolute/relative separation, observer certification limits, or no-container discipline. Any future change requires a migration record and theorem-impact analysis.

RELEASE LABEL V0_OSAP_v1_1_SPECIFICATION_BASELINE_ACCEPTED. Canonical schemas are validated; proof-assistant compilation is a future release gate.

Contents

- Appendices A-E

1. Executive synthesis

v1.1 converts the conceptual architecture of v1.0 into a minimal formal core that can be represented identically across a human specification, JSON records, an executable checker, and proof-assistant libraries. The central requirement is representation fidelity: no implementation may erase the distinctions on which the theory depends.

Human specification <-> *Canonical schemas* <-> *Checker state* <-> *Proof-assistant propositions*

The formal core is intentionally smaller than the full research program. It contains only those constructions required to preserve the program firewalls: typed carriers and registers; live, historical, and retired token partitions; guarded value evaluation; prerequisite closure; dimensional readiness; DLE history; residual obstruction; absolute/relative separation; observer terminal limits; branch typing; and deterministic proof replay.

1.1 v1.1 deliverables

- A finite accepted fragment FC-1 with a closed sort and predicate inventory.
- Canonical registry, proof, evidence, fixture, diagnostic, and release-manifest schemas.
- Deterministic serialization and SHA-256 envelope rules.
- A prioritized diagnostic algebra with no ambiguous PASS on rejected states.
- Lean 4 and Coq correspondence maps for types, records, predicates, and theorem targets.
- A theorem register T121-T150 for the first proof-assistant implementation cycle.
- A canonical positive/countermodel fixture suite and migration discipline.

DESIGN PRINCIPLE The canonical JSON representation is an interchange format, not the semantics itself. Semantic validity is determined by the typed rules and proof obligations, not by schema validity alone.

2. Scope, exclusions, and evidence ladder

2.1 Positive scope

Area	Included in v1.1
Syntax and types	Closed FC-1 sorts, identifiers, contexts, statuses, and judgments.
Registry semantics	State partitions, support families, compatibility, protocols, residual links, observer support, branches.
Decision layer	Deterministic rule ordering, diagnostics, proof status, and replay inputs.
Interchange	Draft 2020-12 JSON Schemas and canonical serialization.
Proof mapping	Lean 4 and Coq representation choices and theorem correspondence obligations.
Release evidence	Validated schemas, fixtures, manifests, checksums, and preflight.

2.2 Exclusions

- No physical dynamics, field equations, metric emergence law, or empirical calibration is introduced.
- No theorem-prover source is claimed compiled or complete in v1.1.
- No schema validation result is promoted to semantic PASS.
- No finite fixture suite is promoted to completeness.
- No absolute V0 claim is derivable from branch-local or observer-relative data.
- No numeric scale is accepted as a replacement for typed status profiles.

2.3 Evidence ladder

Level	Token	Meaning
E0	SPECIFIED	Human-readable rule or schema is defined.
E1	SCHEMA-VALID	Machine record validates against a declared schema version.
E2	CHECKER-REPLAYED	Executable checker deterministically reproduces the result.
E3	MODEL-WITNESSED	Finite model/countermodel supports a fragment-relative claim.
E4	PROOF-ASSISTANT-COMPILED	Lean or Coq accepts the proof under pinned versions.
E5	CROSS-ASSISTANT-REPLAYED	Both mapped backends agree on the stated core

Level	Token	Meaning
		theorem.

v1.1 reaches E1 for the bundled JSON schemas and remains at E0/MAPPED for the Lean 4 and Coq layer. Later releases must report the strongest level actually achieved for each theorem and fixture.

3. Accepted formal fragment FC-1

3.1 Fragment boundary

FC-1 is a finite, first-order, registry-indexed fragment. It permits finite identifier sets, finite support families, Boolean domain and status judgments, finite protocol descriptors, finite branch records, and proof objects whose premises are explicitly enumerated. It excludes unbounded higher-order quantification, unrestricted recursion, real-valued analysis, and physical dynamics.

3.2 Permitted term forms

```
SectorId | NullMarkId | RegisterId | ContextId | StateId | TokenId
ProtocolId | ObserverId | BranchId | EvidenceId | ClaimId | ProofId | RuleId
```

3.3 Permitted judgment forms

```
Typed(x, Sort)
LiveToken(x,rho,A,Sigma) | HistoricalToken(x,rho,A,Sigma) | RetiredToken(x,rho,A,Sigma)
DomHas(x,rho | A,Sigma) | HasVal(x,rho | A,Sigma)=v
Active(S,rho;Sigma) | Ready(M,S;Sigma)
DLE_A(B,rho;Sigma_i,Sigma_j) | LiveResidual(A<-B,rho;Sigma)
RelV0_raw(A<-B;Sigma) | RelV0_robust(A<-B;T,C,Sigma)
Admissible(U;Sigma) | Distinct(U,V;policy) | Supports(e,claim) | Replay(p)=status
```

3.4 Fragment rejection boundary

Input form	FC-1 result
Unguarded numerical value for an undefined protocol	REJECT_VALUE_WITHOUT_LIVE_GUARD.
Open-ended natural-language ontology statement	OUT-OF-SCOPE_UNFORMALIZED.
Infinite branch family without a finite certificate object	DEFERRED_CARDINALITY_CERT.
Recursive support cycle	REJECT_SUPPORT_CYCLE.
Physical interpretation attached without model-assumption type	REJECT_UNTYPED_PHYSICAL_ASSUMPTION.

4. Canonical type universe

4.1 Closed sort inventory

Sort	Canonical role
Sector	Structured or target/reference carrier.
NullMark	Typed null-structural marker; not a Sector.
Register	Structural license category.
Context	Reference index for contextual claims.
RegistryState	Versioned semantic state.
Token	Typed license/status carrier.
Protocol	Versioned measurement or dimensionality procedure.
Observer	Carrier with certification-support registers.
Branch	Admissible structured realization record.
Evidence	Typed support object.
Claim	Normalized proposition identifier.
ProofObject	Replayable derivation record.
Diagnostic	Prioritized checker result.

4.2 Identifier discipline

Every entity MUST have a stable ASCII identifier matching the canonical identifier grammar. Display labels MAY contain Unicode, but semantic references use identifiers only. Identity is never inferred from a display label.

```
identifier := [A-Za-z][A-Za-z0-9_.:-]{0,127}
qualified_id := namespace ":" identifier
version := "v" DIGIT+ "." DIGIT+ ["." DIGIT+]
```

4.3 Disjointness obligations

- NullMark and Sector identifiers are disjoint.
- Live, historical, and retired token sets are pairwise disjoint within one registry state.
- Protocol identifiers are versioned; a new method definition requires a new version.
- Branch labels do not establish branch identity or distinctness.
- Evidence and proof objects are distinct: evidence can support a proof but is not itself a derivation.

4.4 Value domain

FC-1 permits Boolean, bounded integer, rational-string, enum, identifier-reference, and finite-list values. Floating-point numbers are excluded from normative equality. Quantitative values that require exactness are serialized as canonical decimal or rational strings with declared units and uncertainty metadata.

5. Canonical registry-state model

5.1 Registry-state record

```
RegistryState = {
  schema_version, language_version, registry_state_id, predecessor_state_id?,
  declarations, contexts, tokens, prerequisite_families, compatibility_constraints,
  protocols, residual_links, observers, branches, evidence_objects, claims, metadata
}
```

5.2 Token partitions

Partition	Semantic effect	Guard export
live	Current licensed token with active support.	May export a matching live guard.
historical	Certified prior live token retained for DLE provenance.	MUST NOT export a live guard.
retired	Explicitly invalidated token in current state.	MUST NOT export a live guard.
deferred	Candidate token lacking required evidence.	No guard export.

5.3 State transition record

A transition from Sigma_i to Sigma_j is represented by predecessor linkage and explicit token events. The model does not infer time metaphysics from the transition relation; it records a versioned audit order.

```
TokenEvent = introduce | move_live_to_historical | retire | reactivate_fresh
reactivate_fresh(old_token,new_token) requires old_token != new_token
```

5.4 Registry well-formedness

Invariant	Condition
WF-1	Every referenced identifier is declared at the correct sort.
WF-2	Token partitions are pairwise disjoint.
WF-3	Every live token has at least one admissible support path.
WF-4	Historical and retired tokens cannot appear in live_guard_exports.
WF-5	Every predecessor reference resolves or is explicitly genesis.
WF-6	Every evidence reference resolves and has a declared evidence level.
WF-7	Canonical ordering is independent of input file order.

6. Core semantic rules and invariants

6.1 Guard-before-value rule

$$HasVal(x,rho \mid A,Sigma)=v \text{ is admissible only if } LiveToken(x,rho,A,Sigma)$$

The checker first establishes type correctness and a matching live token. Historical evidence, retired tokens, branch labels, or prior-state guards cannot satisfy the current value obligation.

6.2 No-collapse rule

$$\text{not } \mathrm{Dom}X(x) \;\not\Rightarrow\; X(x)=0,\ \text{false},\ \text{empty},\ \text{infinity},\ \text{or absent}$$

6.3 Context preservation

A normalized claim retains its context identifier. Rewriting DomHas(B,rho | A) as DomHas(B,rho) is context erasure and is rejected unless a registered theorem explicitly proves context independence for that exact predicate and fragment.

6.4 State preservation

Every semantic judgment is evaluated in a specific registry state. A claim cannot silently mix premises from different states. Cross-state reasoning requires a transition proof object.

6.5 Integrated invariant set

ID	Invariant
IC-1	Typed terms precede rule application.
IC-2	Live guard precedes value evaluation.
IC-3	Prerequisites are selected, closed, and compatible.
IC-4	Dimensional output requires protocol readiness.
IC-5	DLE requires prior live evidence and current no-license.
IC-6	Residuals are audited independently from target-license exhaustion.
IC-7	Relative or approximate V0 cannot be promoted to absolute V0.
IC-8	Terminal self-exhaustion cannot be same-state self-certified.
IC-9	V0 is not a container and branch labels are not distinctness proofs.
IC-10	Proof replay and diagnostics are deterministic under canonical input.

7. Prerequisite closure and compatibility

7.1 Support-family schema

```
PrerequisiteFamily = {
  family_id, target_register_id, mode: "all_of" | "one_of",
  prerequisite_register_ids, context_policy, evidence_ids, enabled
}
```

7.2 Closure operator

$$\mathrm{Cl}_{\Sigma}(R0) = \text{least } R \text{ superset } R0 \text{ closed under the selected prerequisite families}$$

The closure algorithm is finite in FC-1. A one_of family requires an explicit selected family identifier. The checker never chooses an alternative silently.

7.3 Compatibility constraints

```
CompatibilityConstraint = {
  constraint_id, kind: "forbid_together" | "requires_context" | "requires_evidence",
  register_ids, context_id?, evidence_ids, severity
}
```

7.4 Decision procedure

```
1 collect requested active registers
2 resolve enabled support families
3 require explicit selection for alternatives
4 compute finite transitive closure
5 reject cycles not declared benign
6 apply compatibility constraints
7 emit closure certificate or prioritized diagnostic
```

NON-TEMPORALITY The closure order is proof dependency, not physical time. A later item in the closure calculation is not thereby later in a universe.

8. Dimensional-protocol core

8.1 Protocol record

```
Protocol = {
  protocol_id, protocol_version, carrier_sort, required_register_ids,
  parameter_schema_id?, output_kind, comparison_policy_id, uncertainty_policy_id,
  implementation_ref?, evidence_ids
}
```

8.2 Readiness rule

Ready(M,S;Sigma) iff Typed(M,Protocol) and Typed(S,carrier_sort(M)) and Req(M) subset ActiveRegs(S;Sigma)

8.3 Output statuses

Status	Meaning	Numeric substitution allowed?
READY_VALUE	Protocol ready and a typed output is available.	Yes, within declared output type.
UNDEFINED_DOMAIN	Required structural domain is absent.	No.
INDETERMINATE	Domain exists but evidence is insufficient.	No.
DIVERGENT	Protocol-defined sequence diverges under declared rule.	Only as tagged status.
INCOMPARABLE	Comparison policy does not license an order.	No forced ranking.
RETIRED	Prior protocol license was explicitly retired.	No current value.

8.4 Comparison certificates

Cross-protocol comparison requires a named comparison policy, shared carrier typing, compatible resolution and uncertainty metadata, and an evidence object. Equality of output numbers alone is not a cross-protocol equivalence proof.

9. DLE and residual-obstruction core

9.1 Canonical DLE record

```
DLERecord = {
  dle_id, reference_context_id, target_id, register_id,
  prior_state_id, current_state_id, historical_token_id,
  current_no_license_evidence_id, transition_proof_id
}
```

9.2 DLE rule

DLE_A(B,rho;Sigma_i,Sigma_j) iff WasLive_A(B,rho;Sigma_i) and NoLicense_A(B,rho;Sigma_j)

A historical token is proof of prior status only. It cannot be inserted into the current live-guard set. Reactivation creates a fresh token and does not delete the historical record.

9.3 Residual link record

```
ResidualLink = {
  residual_link_id, boundary_id, carrier_id, residual_register_id,
  link_kind: external | internal | causal, support_path_ids, evidence_ids
}
```

9.4 Obstruction rules

LiveResidual(A<-B,rho) entails not RelV0_raw(A<-B)

LiveResidual(A<-B,rho) and NonElim_T,C(rho) entails not RelV0_robust(A<-B;T,C)

9.5 Diagnostic minimum

- REJECT_DLE_WITHOUT_HISTORY
- REJECT_DLE_WITHOUT_CURRENT_NO_LICENSE
- REJECT_HISTORICAL_GUARD_EXPORT
- REJECT_REACTIVATION_TOKEN_REUSE
- REJECT_LIVE_RESIDUAL_FOR_RAW_NULL
- REJECT_NONELIM_RESIDUAL_FOR_ROBUST_NULL

10. Absolute/relative and approximation firewall

10.1 Typed claim classes

Claim class	Canonical type	Promotion rule
Absolute null marker claim	AbsV0Claim	Only axiomatic/registry-null discourse; never inferred from a local DLE.
Raw relative-null claim	RelV0RawClaim	Requires strong target DLE and empty live-residual set.
Robust relative-null claim	RelV0RobustClaim	Requires every live residual eliminable at declared T,C.
Approximation claim	V0ApproxClaim	Requires metric/topology/protocol and tolerance certificate.
Heuristic score	V0HeuristicClaim	No proof role; must disclose loss and scope.

10.2 Prohibited coercions

```
RelV0RawClaim     -X-> AbsV0Claim
RelV0RobustClaim  -X-> AbsV0Claim
V0ApproxClaim     -X-> AbsV0Claim
V0HeuristicClaim  -X-> any theorem-level V0 claim
```

10.3 Approximation evidence

An approximation claim must name the comparison space, distance or preorder, tolerance, observed components, omitted components, and stability assumptions. Without these fields, the result is DEFERRED_APPROXIMATION_CERT rather than false.

11. Observer-terminal certification core

11.1 Observer support closure

```
ObserverSupport = {
  observer_id, perception_registers, memory_registers, inference_registers,
  communication_registers, identity_registers, proof_registers, validation_registers,
  external_witness_ids, archive_ids, quorum_policy_id?
}
```

11.2 Terminal-limit rule

$$\mathit{StrongSelfDLE\_A(A, SupportClosure(A))} \;\; \mathit{does\ not\ permit} \;\; \mathit{SameStateSelfCert\_A(StrongSelfDLE\_A(A))}$$

The restriction is support-relative: partial self-certification is allowed if the proof object identifies an independent live support path not included in the exhausted closure.

11.3 Archive and witness rules

Object	Allowed role	Prohibited role
Archive	Preserve prior evidence and support later external certification.	Export a current live guard for the exhausted observer.
External witness	Certify a claim under an independence policy.	Count as independent when support is circular or common-mode.
Quorum	Aggregate separately typed witnesses.	Treat duplicated evidence paths as distinct votes.
Identity bridge	Relate observer states under explicit criteria.	Infer identity from label continuity alone.

11.4 Certification horizon

The last-certified frontier is the latest registry state for which the claim has a support path independent of the registers retired by the claim itself. Beyond that frontier the status is DEFERRED_TERMINAL_CERT unless external evidence is present.

12. Plural realization and branch core

12.1 Branch record

```
Branch = {
  branch_id, realization_kernel_id, active_register_ids, protocol_ids,
  local_context_ids, compatibility_certificate_id, closure_certificate_id,
  distinctness_policy_id?, overlap_kernel_ids, translation_ids, evidence_ids
}
```

12.2 Admissibility

Admissible(U;Sigma) iff Typed(U,Branch) and Closed(U) and Compatible(U) and EvidenceComplete(U)

12.3 Distinctness

Distinctness is policy-relative. Allowed policies include non-isomorphism, incompatible law profile, divergent activated-register profile, or certified observational separation. Different branch IDs or labels are never sufficient.

12.4 No-container rule

not DomContains(V0,U) unless a separate typed meta-structure licenses Contains

12.5 Cardinality discipline

A cardinality claim must distinguish registry record count, indexed-copy count, and structural equivalence-class count. Claims beyond finite enumerated records require a typed meta-index and proof evidence; otherwise the status is DEFERRED_CARDINALITY_CERT.

13. Status algebra and diagnostic precedence

13.1 Result statuses

Rank	Status	Meaning
0	REJECT	A decisive violation in the accepted fragment.
1	DEFERRED	Required certificate is absent or below threshold.
2	INDETERMINATE	Well-typed evidence does not decide the requested relation.
3	OUT-OF-SCOPE	Input is not expressible in FC-1.
4	PASS	All required checks in FC-1 passed at the reported evidence level.

PASS is emitted only when no higher-priority rejection, deferral, indeterminacy, or scope boundary applies. Implementations must not treat OUT-OF-SCOPE as PASS.

13.2 Diagnostic ordering

```
TYPE > STATE > GUARD > PREREQUISITE > COMPATIBILITY > PROTOCOL > DLE > RESIDUAL > FIREWALL > OBSERVER >
BRANCH > PROOF > RELEASE
```

13.3 Canonical diagnostic object

```
Diagnostic = {
  code, status, priority, message_template_id, instance_path,
  related_ids, evidence_ids, registry_state_id, implementation_version
}
```

13.4 Stability requirement

For a fixed canonical input, semantic version, and implementation version, the primary status and ordered diagnostic code list must be deterministic. Message wording may vary only outside the canonical message-template field.

14. Canonical schema architecture

14.1 Bundle inventory

Schema file	Purpose
common.schema.json	Identifiers, versions, statuses, evidence levels, and reusable definitions.
registry_state.schema.json	Complete FC-1 registry-state interchange record.
proof_object.schema.json	Replayable proof-object representation.
evidence_object.schema.json	Typed evidence with provenance and level.
fixture.schema.json	Positive, countermodel, and mutation-test fixtures.
diagnostic.schema.json	Canonical checker diagnostic.
release_manifest.schema.json	File hashes, versions, checker and backend metadata.

14.2 Schema principles

- JSON Schema Draft 2020-12 is the normative schema dialect.
- Unknown properties are rejected at normative object boundaries unless an explicit extensions object is present.
- Identifiers are references, not embedded duplicate semantic objects.
- Enums are closed within a schema version.
- Every record carries schema_version and semantic_version where behavior depends on rules.
- Migrations are explicit transformations, never silent parser coercions.

14.3 Example token object

```
{
  "token_id": "tok:B:rho.pres:A:0001",
  "carrier_id": "sector:B",
  "register_id": "reg:presence",
  "context_id": "ctx:A",
  "partition": "historical",
  "introduced_in": "state:001",
  "moved_in": "state:002",
  "source_evidence_id": "ev:hist:0001"
}
```

SCHEMA LIMIT A record can be schema-valid while semantically invalid, for example when a historical token is referenced as a live guard. Semantic checker rules remain mandatory.

15. Deterministic serialization and hash envelope

15.1 Canonical JSON rules

- UTF-8 without BOM.
- Object keys sorted lexicographically by Unicode code point; all semantic identifiers are ASCII.
- No insignificant whitespace in the hashed representation.
- Arrays are sorted only when declared order-insensitive by the schema mapping; proof-premise order remains preserved when rule-significant.
- Numbers are forbidden for values requiring exact cross-runtime identity; use canonical strings.
- Line endings are LF in source artifacts.

15.2 Hash envelope

```
HashEnvelope = {
  algorithm: "sha256", canonicalization: "V0-OSAP-CJ-1",
  payload_schema_id, payload_schema_version, payload_hash, byte_length
}
```

15.3 Traceability theorem boundary

Equal hashes under the same canonicalization identify equal canonical bytes. They do not prove semantic correctness, provenance truth, theorem soundness, or physical validity.

15.4 Round-trip requirement

$$parse(serialize\_canonical(x)) = x \quad \text{for every well-formed FC-1 object}$$

16. Proof objects and evidence objects

16.1 Proof-object schema

```
ProofObject = {
  proof_id, claim_id, rule_id, premise_claim_ids, registry_state_id, context_id,
  selected_support_family_ids, translation_class?, conservativity_level?,
  evidence_ids, implementation_version, proof_status, replay_hash
}
```

16.2 Evidence-object schema

```
EvidenceObject = {
  evidence_id, evidence_type, evidence_level, source_uri?, source_hash?,
  produced_by, produced_at?, registry_state_id?, supports_claim_ids,
  independence_group_id?, limitations, metadata
}
```

16.3 Evidence monotonicity

A later artifact may upgrade an evidence level only by adding the required stronger object. It may not relabel schema validation as theorem proof. Downgrades caused by stale dependencies or broken provenance must be visible.

16.4 Replay contract

```
Replay(proof, registry, ruleset) -> {status, ordered_diagnostics, derived_claims, replay_hash}
Required: deterministic under pinned inputs; no network access; no hidden registry state.
```

17. Lean 4 mapping

17.1 Representation choices

FC-1 concept	Lean 4 target
Identifiers	abbrev Id := String with WellFormedId predicate.
Closed enums	inductive SortTag / TokenPartition / ResultStatus.
Registry record	structure RegistryState with Finset-based fields and wellFormed proposition.
Predicates	Prop-valued definitions indexed by RegistryState.
Decidable checks	Boolean checker plus theorem connecting check = true to Prop.
Finite closure	Finset fixed-point iteration with termination measure.
Proof objects	structure ProofObject; replay as total function returning Result.

17.2 Mapping sketch

```
inductive TokenPartition | live | historical | retired | deferred
structure Token where
  tokenId : String
  carrierId : String
  registerId : String
  contextId : String
  partition : TokenPartition

def exportsGuard (t : Token) : Prop := t.partition = .live

theorem historical_no_guard (t : Token)
  (h : t.partition = .historical) : not exportsGuard t := by ...
```

17.3 Lean proof obligations

- Prove decidability and soundness for identifier/type checks.
- Prove pairwise disjoint token partitions from wellFormed.
- Prove closure termination and closure soundness for finite support graphs.
- Prove semantic checker soundness for each rejection firewall.
- Prove deterministic replay under canonical inputs.
- Export theorem names matching the canonical theorem register.

17.4 Lean namespace policy

```
V0OSAP.Core.Types
V0OSAP.Core.Registry
V0OSAP.Core.Guards
V0OSAP.Core.Closure
V0OSAP.Core.Dimension
V0OSAP.Core.DLE
V0OSAP.Core.Observer
V0OSAP.Core.Branch
V0OSAP.Core.Replay
```

18. Coq mapping

18.1 Representation choices

FC-1 concept	Coq target
Identifiers	string plus valid_id predicate and decidability lemma.
Closed enums	Inductive sort_tag, token_partition, result_status.
Registry record	Record registry_state with list-based finite fields and NoDup invariants.
Predicates	Prop definitions indexed by registry_state.
Executable checker	bool functions with reflect or soundness lemmas.
Finite closure	fuel-bounded or well-founded recursion over finite register lists.
Proof objects	Record proof_object and deterministic replay function.

18.2 Mapping sketch

```
Inductive token_partition := Live | Historical | Retired | Deferred.
Record token := {
  token_id : string; carrier_id : string; register_id : string;
  context_id : string; partition : token_partition
}.
Definition exports_guard (t : token) : Prop := partition t = Live.
Theorem historical_no_guard : forall t, partition t = Historical -> ~ exports_guard t.
Proof. ... Qed.
```

18.3 Coq proof obligations

- Provide equality decision procedures for every closed identifier wrapper and enum.
- Connect list membership checks to Prop membership.
- Prove NoDup and partition-disjointness consequences of well_formed_registry.
- Prove closure algorithm soundness relative to the relational closure definition.
- Prove diagnostic precedence and replay determinism.
- Maintain theorem identifiers aligned with the v1.1 theorem register.

18.4 Module policy

```
V0OSAP_Types.v
V0OSAP_Registry.v
V0OSAP_Guards.v
V0OSAP_Closure.v
V0OSAP_Dimension.v
V0OSAP_DLE.v
V0OSAP_Observer.v
V0OSAP_Branch.v
V0OSAP_Replay.v
```

19. Cross-assistant correspondence obligations

19.1 Correspondence boundary

The two proof assistants need not use identical data structures. They must implement extensionally equivalent FC-1 propositions and report the same theorem identifiers, assumptions, and accepted-fragment boundary.

19.2 Correspondence record

```
AssistantMapping = {
  theorem_id, canonical_statement_hash, lean_name, coq_name,
  assumption_ids, schema_version, semantic_version, status_lean, status_coq
}
```

19.3 Required comparison checks

Check	Requirement
Statement identity	Canonical normalized statement hash matches.
Assumptions	Same explicit axiom/rule IDs; no hidden backend-specific axiom.
Finite semantics	Equivalent treatment of duplicates, ordering, and membership.
Diagnostics	Same primary status and canonical code for fixtures.
Evidence label	Compiler status reported independently for each backend.

19.4 Permitted divergence

Proof terms, libraries, recursion strategies, and internal representations may differ. A divergence is acceptable only when the canonical proposition, assumptions, and fixture outcomes remain equivalent and the mapping ledger records the implementation difference.

20. Theorem target register T121-T150

ID	Name	Target statement
T121	Typed-term uniqueness	A semantic identifier has one declared FC-1 sort per registry state.
T122	Token-partition disjointness	Live, historical, retired, and deferred token sets are pairwise disjoint.
T123	Guard-before-value soundness	Accepted value claims have a matching current live token.

ID	Name	Target statement
T124	Historical non-export	Historical tokens cannot establish current DomHas.
T125	Retired non-export	Retired tokens cannot establish current DomHas.
T126	Finite closure existence	FC-1 prerequisite closure exists for finite declared graphs.
T127	Closure minimality	Computed closure is the least prerequisite-closed superset.
T128	Alternative-support transparency	Every accepted one_of obligation records its selected family.
T129	Compatibility preservation	Accepted activation profiles satisfy all enabled compatibility constraints.
T130	Dimensional readiness soundness	READY_VALUE implies all protocol prerequisites are active.
T131	Undefined is not zero	UNDEFINED_DOMAIN cannot be coerced to a numeric dimension.
T132	DLE history adequacy	Accepted DLE has prior live evidence and current no-license.
T133	Fresh-token reactivation	Reactivation cannot reuse the historical/retired token identifier.
T134	Raw residual obstruction	Any live residual blocks raw relative nullity.
T135	Robust residual obstruction	A live non-eliminable residual blocks robust relative nullity.
T136	Relative-to-absolute non-promotion	No FC-1 rule promotes relative V0 to absolute V0.
T137	Approximation non-identity	Approximation certificates do not entail V0 identity.
T138	Terminal self-certification limit	Same-state exhausted support cannot certify its own total exhaustion.
T139	Archive non-guard-export	An archive preserves evidence but does not create a current observer guard.
T140	Independent-witness conditional sufficiency	A policy-compliant independent witness may support an external certificate.
T141	No-container	V0 has no ordinary Contains relation to a branch in FC-1.
T142	Branch-label insufficiency	Different labels do not prove branch distinctness.
T143	Cardinality licensing	Non-finite cardinality claims require a typed meta-index and evidence.
T144	Diagnostic precedence totality	Every finite diagnostic set has a unique primary status under the precedence order.
T145	Canonical serialization determinism	A well-formed object has one V0-OSAP-CJ-1 byte representation.
T146	Round-trip identity	Canonical serialize/parse preserves well-formed FC-1 objects.
T147	Replay determinism	Pinned proof, registry, and ruleset produce one canonical replay result.
T148	Schema migration visibility	A semantic-version change cannot be hidden as parser coercion.
T149	Backend statement correspondence	Mapped Lean and Coq theorem entries share a canonical statement hash.
T150	Accepted-fragment checker soundness	Conditional on proved rule lemmas, checker PASS implies the FC-1 semantic obligations hold.

PROOF STATUS T121-T150 are theorem targets in v1.1. They are not claimed proof-assistant-complete until a later release supplies compiler evidence and mapping records.

21. Canonical fixture corpus

21.1 Positive fixtures

ID	Purpose	Expected
P11-01	Minimal well-typed live guard and value.	PASS
P11-02	Conjunctive prerequisite closure.	PASS
P11-03	Explicit one_of support selection.	PASS

ID	Purpose	Expected
P11-04	Ready dimensional protocol with rational-string output.	PASS
P11-05	DLE with historical token and current no-license.	PASS
P11-06	Target DLE with independent eliminable residual.	PASS robust policy-specific
P11-07	External observer witness with independence certificate.	PASS
P11-08	Two branch records with certified non-isomorphism.	PASS

21.2 Countermodels and mutation fixtures

ID	Defect	Expected diagnostic
N11-01	Value without live guard.	REJECT_VALUE_WITHOUT_LIVE_GUARD
N11-02	Historical token exported as live.	REJECT_HISTORICAL_GUARD_EXPORT
N11-03	Missing prerequisite family.	REJECT_PREREQUISITE_GAP
N11-04	Dimension emitted under UNDEFINED_DOMAIN.	REJECT_DIMENSION_WITHOUT_PROTOCOL_READY
N11-05	DLE without prior live evidence.	REJECT_DLE_WITHOUT_HISTORY
N11-06	Raw null with live residual.	REJECT_LIVE_RESIDUAL_FOR_RAW_NULL
N11-07	Relative-to-absolute promotion.	REJECT_RELATIVE_TO_ABSOLUTE_PROMOTION
N11-08	Terminal same-state self-certification.	REJECT_TERMINAL_SELF_CERTIFICATION
N11-09	V0 used as branch container.	REJECT_V0_AS_CONTAINER
N11-10	Distinctness inferred from labels only.	REJECT_BRANCH_LABEL_AS_DISTINCTNESS
N11-11	Non-canonical exact number serialized as float.	REJECT_NONCANONICAL_NUMBER
N11-12	Proof replay hash mismatch.	REJECT_REPLAY_HASH_MISMATCH

21.3 Oracle independence

Expected results are stored separately from checker output and reviewed against the human rule ledger. Updating an expected outcome requires a change rationale, theorem-impact note, and regression review.

22. Implementation architecture

```
v0-osap-formal-core/
  schemas/
    common.schema.json
    registry_state.schema.json
    proof_object.schema.json
    evidence_object.schema.json
    fixture.schema.json
    diagnostic.schema.json
    release_manifest.schema.json
  semantics/
    rules.yaml
    diagnostics.yaml
    theorem_register.yaml
  checker/
  fixtures/positive/
  fixtures/negative/
  mappings/lean4/
  mappings/coq/
  proofs/lean4/
  proofs/coq/
  oracle/
  ci/
  docs/
```

22.1 Small trusted core

The executable trusted core should be small: parser, schema validator, canonicalizer, type checker, registry well-formedness checker, semantic rule engine, proof replay, and manifest writer. Rendering, reporting, and user-interface layers are outside the trusted semantic core.

22.2 Extension policy

Extensions enter through namespaced records and cannot modify core enums or rule meaning without a semantic-version change. An implementation that ignores an unknown normative field must reject the record rather than continue silently.

23. Verification and CI plan

23.1 CI stages

Stage	Required checks
S1 Schema	Validate all schemas against metaschema; validate examples and fixtures.
S2 Canonicalization	Round-trip and stable-hash tests across at least two JSON runtimes.
S3 Checker	Positive/negative fixtures; mutation coverage; diagnostic order.
S4 Lean 4	Pinned compiler, no sorry/admit in release theorem files.
S5 Coq	Pinned compiler, no Admitted/Axiom beyond declared AX0-AX7 bridge.
S6 Correspondence	Statement hashes, assumption ledgers, and fixture status agreement.
S7 Release	Manifest, SHA-256, provenance, documentation, and reproducible archive.

23.2 Failure policy

A release is blocked by schema drift, non-deterministic canonical bytes, fixture disagreement, hidden axioms, proof placeholders, unmatched theorem hashes, or missing provenance. A backend may be marked DEFERRED, but the release must not advertise cross-assistant completion.

23.3 Reproducibility record

```
compiler versions | dependency lock hashes | OS/container digest | git commit
schema bundle hash | fixture corpus hash | theorem mapping hash | release manifest hash
```

24. Migration and interoperability

24.1 Migration from v1.0

v1.0 concept	v1.1 canonicalization
Typed language and registry state	Closed FC-1 sorts and registry_state.schema.json.
Master validation pipeline	Deterministic rule order and diagnostic precedence.
Proof/evidence objects	Separate proof_object and evidence_object schemas.
Mechanization roadmap	Lean 4 and Coq mapping obligations plus theorem register.
Model corpus	Versioned fixture schema and independent oracle.
Release discipline	Canonical serialization and release manifest.

24.2 Relationship to V0 Validator Core

The archived V0 Validator Core v0.12 remains a separate compiler-passed artifact for the earlier domain-guard/DLE core. v1.1 defines a new implementation line because dimensional protocols, observer reflexivity, branch semantics, canonical schemas, and cross-assistant correspondence exceed the closed release scope. Reuse is permitted only through documented adapters and theorem mappings.

24.3 Compatibility labels

Label	Meaning
READS-v1.1	Parser accepts the v1.1 schemas.
CHECKS-FC1-v1.1	Checker implements all FC-1 semantic rules.
PROVES-LEAN-v1.1	Lean target set compiler-passed under pinned version.
PROVES-COQ-v1.1	Coq target set compiler-passed under pinned version.
CROSS-REPLAY-v1.1	Both theorem maps and canonical fixtures agree.

25. Release gates

Gate	Area	Pass condition
G11-1	Axiom freeze	AX0-AX7 unchanged and explicitly imported.
G11-2	Fragment closure	Every accepted syntax and status token is enumerated.
G11-3	Schema validity	All bundled schemas and examples validate.
G11-4	Serialization	Canonicalization rules and hash envelope are fixed.
G11-5	Diagnostic determinism	Status precedence has no unresolved tie.
G11-6	Theorem register	T121-T150 have unique statements and backend names reserved.
G11-7	Fixture adequacy	Each firewall has at least one positive or admissible case and one decisive negative case.
G11-8	Evidence honesty	No compiler-passed label without compiler evidence.
G11-9	Migration visibility	Schema and semantic changes require explicit versioning.
G11-10	Package integrity	DOCX, PDF, schema bundle, manifest, preflight, and hashes agree.

26. Risks and limitations

Risk	Impact	Mitigation
Schema/semantics confusion	Schema-valid invalid states may be accepted socially.	Prominent evidence ladder and mandatory semantic checker.
Backend mismatch	Lean and Coq formalize subtly different propositions.	Canonical statement hashes and assumption ledgers.
Hidden axioms	Proof compiles by importing undeclared assumptions.	Pinned import list and axiom audit.
Finite-model overconfidence	Fixture success mistaken for general proof.	Separate MODEL-WITNESSED and PROOF-ASSISTANT-COMPILED.
Canonicalization ambiguity	Hash differs across runtimes.	V0-OSAP-CJ-1 tests in multiple runtimes.
State-context erasure	Claims combine incompatible registry states.	State IDs mandatory in judgments and proof objects.
Premature physical reading	Formal activation described as cosmological generation.	Non-claim perimeter and typed model assumptions.
Specification bloat	Trusted core becomes unauditable.	Keep FC-1 finite and extension mechanisms namespaced.

27. v1.1 acceptance ledger

27.1 Completed items

- FC-1 accepted-fragment boundary specified.
- Canonical type, registry, proof, evidence, fixture, diagnostic, and manifest records specified.
- Deterministic serialization and hash envelope specified.
- Core rule and diagnostic precedence specified.
- Lean 4 and Coq design mappings registered.
- Theorem targets T121-T150 registered.
- Canonical JSON Schema bundle generated and locally validated.
- Positive and negative fixture plan registered.

27.2 Deferred items

- Executable semantic checker implementation.
- Lean 4 compilation of T121-T150.
- Coq compilation of T121-T150.
- Cross-assistant theorem replay and statement-hash audit.
- CI-backed public repository release and archival DOI.

27.3 Final decision

ACCEPTED BASELINE V0_OSAP_v1_1_SPECIFICATION_BASELINE_ACCEPTED. The schema layer is SCHEMA-VALID. Proof-

assistant mappings are MAPPED but not COMPILER-PASSED.

27.4 Recommended next step

Proceed to V0_OSAP v1.2 - Executable FC-1 Checker, Canonical Fixture Corpus, and Dual-Backend Proof Skeleton. The first implementation milestone should prove T121-T135, because those theorems protect typing, guard discipline, closure, dimensional readiness, DLE history, and residual obstruction.

Appendix A. Canonical schema field map

Object	Required identity fields	Required semantic fields
RegistryState	schema_version, language_version, registry_state_id	declarations, contexts, tokens, prerequisites, compatibility, protocols, residual_links, observers, branches, evidence_objects, claims
ProofObject	proof_id, claim_id, rule_id, registry_state_id	premises, selected supports, evidence_ids, proof_status, replay_hash
EvidenceObject	evidence_id, evidence_type, evidence_level	producer, supported claims, limitations, provenance
Fixture	fixture_id, kind, schema_version, semantic_version	input, expected status, expected diagnostics, theorem targets, oracle source
Diagnostic	code, status, priority	instance path, related IDs, evidence IDs, implementation version
ReleaseManifest	release_id, semantic_version	file hashes, checker/backend versions, theorem mapping, status

Appendix B. Canonical diagnostic catalogue

Code	Family
REJECT_UNTYPED_TERM	Type
REJECT_DUPLICATE_SORT_DECLARATION	Type
REJECT_STALE_REGISTRY_STATE	State
REJECT_CONTEXT_ERASURE	State
REJECT_VALUE_WITHOUT_LIVE_GUARD	Guard
REJECT_HISTORICAL_GUARD_EXPORT	Guard
REJECT_RETIRED_GUARD_EXPORT	Guard
REJECT_PREREQUISITE_GAP	Prerequisite
REJECT_SUPPORT_CYCLE	Prerequisite
REJECT_UNSELECTED_ALTERNATIVE	Prerequisite
REJECT_INCOMPATIBLE_ACTIVATION	Compatibility
REJECT_DIMENSION_WITHOUT_PROTOCOL_READY	Protocol
REJECT_NONCANONICAL_NUMBER	Protocol/serialization
REJECT_DLE_WITHOUT_HISTORY	DLE
REJECT_DLE_WITHOUT_CURRENT_NO_LICENSE	DLE
REJECT_REACTIVATION_TOKEN_REUSE	DLE
REJECT_LIVE_RESIDUAL_FOR_RAW_NULL	Residual
REJECT_NONELIM_RESIDUAL_FOR_ROBUST_NULL	Residual
REJECT_RELATIVE_TO_ABSOLUTE_PROMOTION	Firewall
REJECT_APPROXIMATION_AS_IDENTITY	Firewall
REJECT_TERMINAL_SELF_CERTIFICATION	Observer
REJECT_CIRCULAR_QUORUM	Observer
REJECT_IDENTITY_LABEL_ONLY	Observer
REJECT_V0_AS_CONTAINER	Branch
REJECT_BRANCH_LABEL_AS_DISTINCTNESS	Branch
REJECT_CARDINALITY_WITHOUT_INDEX	Branch

Code	Family
REJECT_REPLAY_HASH_MISMATCH	Proof
REJECT_SCHEMA_SEMANTIC_VERSION_MISMATCH	Release
DEFERRED_CERT	General
INDETERMINATE_COMPARISON	General
OUT_OF_SCOPE_FC1	General
PASS_ACCEPTED_FRAGMENT	General

Appendix C. Lean 4 / Coq theorem-name reservation

ID	Lean 4 reserved name	Coq reserved name
T121	V0OSAP.Core.typed_term_uniqueness	V0OSAP_typed_term_uniqueness
T122	V0OSAP.Core.token_partition_disjointness	V0OSAP_token_partition_disjointness
T123	V0OSAP.Core.guard_before_value_soundness	V0OSAP_guard_before_value_soundness
T124	V0OSAP.Core.historical_non_export	V0OSAP_historical_non_export
T125	V0OSAP.Core.retired_non_export	V0OSAP_retired_non_export
T126	V0OSAP.Core.finite_closure_existence	V0OSAP_finite_closure_existence
T127	V0OSAP.Core.closure_minimality	V0OSAP_closure_minimality
T128	V0OSAP.Core.alternative_support_transparency	V0OSAP_alternative_support_transparency
T129	V0OSAP.Core.compatibility_preservation	V0OSAP_compatibility_preservation
T130	V0OSAP.Core.dimensional_readiness_soundness	V0OSAP_dimensional_readiness_soundness
T131	V0OSAP.Core.undefined_is_not_zero	V0OSAP_undefined_is_not_zero
T132	V0OSAP.Core.dle_history_adequacy	V0OSAP_dle_history_adequacy
T133	V0OSAP.Core.fresh_token_reactivation	V0OSAP_fresh_token_reactivation
T134	V0OSAP.Core.raw_residual_obstruction	V0OSAP_raw_residual_obstruction
T135	V0OSAP.Core.robust_residual_obstruction	V0OSAP_robust_residual_obstruction
T136	V0OSAP.Core.relative_to_absolute_non_promotion	V0OSAP_relative_to_absolute_non_promotion
T137	V0OSAP.Core.approximation_non_identity	V0OSAP_approximation_non_identity
T138	V0OSAP.Core.terminal_self_certification_limit	V0OSAP_terminal_self_certification_limit
T139	V0OSAP.Core.archive_non_guard_export	V0OSAP_archive_non_guard_export
T140	V0OSAP.Core.independent_witness_conditional_sufficiency	V0OSAP_independent_witness_conditional_sufficiency
T141	V0OSAP.Core.no_container	V0OSAP_no_container
T142	V0OSAP.Core.branch_label_insufficiency	V0OSAP_branch_label_insufficiency
T143	V0OSAP.Core.cardinality_licensing	V0OSAP_cardinality_licensing
T144	V0OSAP.Core.diagnostic_precedence_totality	V0OSAP_diagnostic_precedence_totality
T145	V0OSAP.Core.canonical_serialization_determinism	V0OSAP_canonical_serialization_determinism
T146	V0OSAP.Core.round_trip_identity	V0OSAP_round_trip_identity
T147	V0OSAP.Core.replay_determinism	V0OSAP_replay_determinism
T148	V0OSAP.Core.schema_migration_visibility	V0OSAP_schema_migration_visibility
T149	V0OSAP.Core.backend_statement_correspondence	V0OSAP_backend_statement_correspondence
T150	V0OSAP.Core.accepted_fragment_checker_soundness	V0OSAP_accepted_fragment_checker_soundness

Appendix D. Bundle manifest

- README.md
- common.schema.json
- registry_state.schema.json
- proof_object.schema.json
- evidence_object.schema.json
- fixture.schema.json
- diagnostic.schema.json

- release_manifest.schema.json
- canonical_example_registry.json
- canonical_example_proof.json
- lean4_mapping.md
- coq_mapping.md
- schema_manifest.json

Appendix E. Final handoff

v1.1 establishes a stable interchange and theorem-mapping boundary. v1.2 should avoid expanding the ontology until the FC-1 checker and the first theorem subset are implemented. Any new semantic feature should enter only after the existing canonical fixtures replay identically and schema migration is automated.